Blog Platform with Comments

Complete Solution for Task 4

A full-stack blogging platform where authenticated users can create, edit and delete posts and interact through comments.

Requirement	Implemented
Registration/Login/Authentication	Yes
Create/Edit/Delete Posts	Yes
Comments & User Interaction	Yes
RESTful APIs	Yes
Database Integration	SQLite
Responsive Design	Yes
Ownership Authorization	Yes

How to run

1. Extract the ZIP and open the folder in VS Code.
2. Open Terminal.
3. Run **npm install**.
4. Run **npm start**.
5. Open **http://localhost:3000**.
6. Register/login, create a post and add comments.

Project structure

```
blog-platform-with-comments/
■■■ package.json
■■■ server.js
■■■ README.md
■■■ public/
    ■■■ index.html
    ■■■ style.css
    ■■■ script.js
```

package.json

```
{
  "name":"blog-platform-with-comments",
  "version":"1.0.0",
  "scripts":{"start":"node server.js"},
  "dependencies":{
    "bcryptjs":"^2.4.3",
    "better-sqlite3":"^11.3.0",
    "cors":"^2.8.5",
    "express":"^4.21.0",
    "jsonwebtoken":"^9.0.2"
  }
}
```

server.js

```
const express=require("express");
const cors=require("cors");
const path=require("path");
const bcrypt=require("bcryptjs");
const jwt=require("jsonwebtoken");
const Database=require("better-sqlite3");

const app=express(), PORT=3000, SECRET="change_this_secret_in_production";
const db=new Database("blog.db");
db.pragma("foreign_keys=ON");

db.exec(`
CREATE TABLE IF NOT EXISTS users(
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 name TEXT NOT NULL,
 email TEXT UNIQUE NOT NULL,
 password TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS posts(
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 title TEXT NOT NULL,
 content TEXT NOT NULL,
 user_id INTEGER NOT NULL,
 created_at TEXT DEFAULT CURRENT_TIMESTAMP,
 updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY(user_id) REFERENCES users(id) ON DELETE CASCADE
);
CREATE TABLE IF NOT EXISTS comments(
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 content TEXT NOT NULL,
 post_id INTEGER NOT NULL,
 user_id INTEGER NOT NULL,
 created_at TEXT DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY(post_id) REFERENCES posts(id) ON DELETE CASCADE,
 FOREIGN KEY(user_id) REFERENCES users(id) ON DELETE CASCADE
);
`);

app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname,"public")));

function auth(req,res,next){
  const h=req.headers.authorization;
  if(!h)return res.status(401).json({message:"Login required"});
  try{req.user=jwt.verify(h.split(" ")[1],SECRET);next();}
  catch{res.status(401).json({message:"Invalid or expired token"});}
}

app.post("/api/register",async(req,res)=>{
  const {name,email,password}=req.body;
  if(!name||!email||!password)return res.status(400).json({message:"All fields are required"});
  try{
    const hash=await bcrypt.hash(password,10);
    const r=db.prepare("INSERT INTO users(name,email,password) VALUES(?,?,?)")
      .run(name.trim(),email.trim().toLowerCase(),hash);
    res.status(201).json({message:"Registration successful",id:r.lastInsertRowid});
  }catch(e){res.status(409).json({message:"Email already registered"});}
});

app.post("/api/login",async(req,res)=>{
  const {email,password}=req.body;
  const u=db.prepare("SELECT * FROM users WHERE email=?").get(email.trim().toLowerCase());
  if(!u||!(await bcrypt.compare(password,u.password)))
    return res.status(401).json({message:"Invalid email or password"});
  const token=jwt.sign({id:u.id,name:u.name,email:u.email},SECRET,{expiresIn:"2h"});
  res.json({token,user:{id:u.id,name:u.name,email:u.email}});
});

app.get("/api/posts",(req,res)=>{
  const posts=db.prepare(`
    SELECT posts.id,posts.title,posts.content,posts.user_id,posts.created_at,
           users.name AS author
    FROM posts JOIN users ON users.id=posts.user_id
    ORDER BY posts.id DESC
  `).all();
  res.json(posts);
});

app.get("/api/posts/:id",(req,res)=>{
  const post=db.prepare(`
    SELECT posts.*,users.name AS author FROM posts
```

```
    JOIN users ON users.id=posts.user_id WHERE posts.id=?
  `).get(req.params.id);
  if(!post)return res.status(404).json({message:"Post not found"});
  const comments=db.prepare(`
    SELECT comments.id,comments.content,comments.created_at,users.name AS author
    FROM comments JOIN users ON users.id=comments.user_id
    WHERE comments.post_id=? ORDER BY comments.id ASC
  `).all(req.params.id);
  res.json({post,comments});
});

app.post("/api/posts",auth,(req,res)=>{
  const {title,content}=req.body;
  if(!title||!content)return res.status(400).json({message:"Title and content are required"});
  const r=db.prepare("INSERT INTO posts(title,content,user_id) VALUES(?,?,?)")
    .run(title.trim(),content.trim(),req.user.id);
  res.status(201).json({message:"Post created",id:r.lastInsertRowid});
});

app.put("/api/posts/:id",auth,(req,res)=>{
  const {title,content}=req.body;
  const r=db.prepare(`
    UPDATE posts SET title=?,content=?,updated_at=CURRENT_TIMESTAMP
    WHERE id=? AND user_id=?
  `).run(title?.trim(),content?.trim(),req.params.id,req.user.id);
  if(!r.changes)return res.status(404).json({message:"Post not found or not owned by you"});
  res.json({message:"Post updated"});
});

app.delete("/api/posts/:id",auth,(req,res)=>{
  const r=db.prepare("DELETE FROM posts WHERE id=? AND user_id=?")
    .run(req.params.id,req.user.id);
  if(!r.changes)return res.status(404).json({message:"Post not found or not owned by you"});
  res.json({message:"Post deleted"});
});

app.post("/api/posts/:id/comments",auth,(req,res)=>{
  const {content}=req.body;
  if(!content||!content.trim())return res.status(400).json({message:"Comment cannot be empty"});
  const post=db.prepare("SELECT id FROM posts WHERE id=?").get(req.params.id);
  if(!post)return res.status(404).json({message:"Post not found"});
  const r=db.prepare("INSERT INTO comments(content,post_id,user_id) VALUES(?,?,?)")
    .run(content.trim(),req.params.id,req.user.id);
  res.status(201).json({message:"Comment added",id:r.lastInsertRowid});
});

app.delete("/api/comments/:id",auth,(req,res)=>{
  const r=db.prepare("DELETE FROM comments WHERE id=? AND user_id=?")
    .run(req.params.id,req.user.id);
  if(!r.changes)return res.status(404).json({message:"Comment not found or not owned by you"});
  res.json({message:"Comment deleted"});
});

app.get("*",(req,res)=>res.sendFile(path.join(__dirname,"public","index.html")));
app.listen(PORT,()=>console.log(`Blog Platform running at http://localhost:${PORT}`));
```

public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Blog Platform</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<header>
  <h1>MyBlog</h1>
  <nav>
    <button onclick="home()">Home</button>
    <button onclick="openCreate()">Write Post</button>
    <button id="loginBtn" onclick="openAuth()">Login</button>
    <button onclick="logout()">Logout</button>
  </nav>
</header>

<main>
<section id="auth" class="panel hidden">
  <h2 id="authTitle">Login</h2>
  <input id="name" class="registerOnly hidden" placeholder="Name">
  <input id="email" type="email" placeholder="Email">
  <input id="password" type="password" placeholder="Password">
  <button id="authAction" onclick="submitAuth()">Login</button>
  <button class="link" onclick="toggleAuth()">Create account</button>
  <p id="msg"></p>
</section>

<section id="create" class="panel hidden">
  <h2>Create Blog Post</h2>
  <input id="postTitle" placeholder="Post title">
  <textarea id="postContent" placeholder="Write your blog content..."></textarea>
  <button onclick="createPost()">Publish Post</button>
</section>

<section id="feed">
  <h2>Latest Posts</h2>
  <div id="posts"></div>
</section>

<section id="single" class="panel hidden">
  <button onclick="home()">← Back</button>
  <div id="postDetail"></div>
</section>
</main>
<script src="script.js"></script>
</body>
</html>
```

public/style.css

```
*{box-sizing:border-box}
body{margin:0;font-family:Arial,sans-serif;background:#f4f7fb;color:#172033}
header{background:#2563eb;color:white;padding:16px 5%;display:flex;justify-content:space-between;align-items
header h1{margin:0}nav{display:flex;gap:7px;flex-wrap:wrap}
button{border:0;border-radius:8px;padding:10px 14px;background:#2563eb;color:white;font-weight:bold;cursor:p
header button{background:white;color:#2563eb}
main{width:min(950px,94%);margin:25px auto}
.post,.panel{background:white;padding:20px;border-radius:14px;margin:15px 0;box-shadow:0 4px 15px rgba(0,0,0
.post h3{margin-top:0;color:#1d4ed8;cursor:pointer}.meta{color:#667085;font-size:13px}
input,textarea{width:100%;padding:12px;margin:7px 0;border:1px solid #ccd2dc;border-radius:8px;font-size:15p
textarea{min-height:180px;resize:vertical}.comment{border-top:1px solid #e5e7eb;padding:12px 0}
.hidden{display:none!important}.link{background:transparent;color:#2563eb;display:block}
.danger{background:#dc2626}.edit{background:#059669}
@media(max-width:600px){header{padding:14px 3%}main{width:94%}.post,.panel{padding:16px}}
```

public/script.js

```
let posts=[],register=false;

async function home(){
  document.getElementById("create").classList.add("hidden");
  document.getElementById("single").classList.add("hidden");
  document.getElementById("auth").classList.add("hidden");
  document.getElementById("feed").classList.remove("hidden");
  await loadPosts();
}
async function loadPosts(){
  const r=await fetch("/api/posts");posts=await r.json();
  document.getElementById("posts").innerHTML=posts.length?posts.map(p=>`
    <article class="post">
      <h3 onclick="viewPost(${p.id})">${esc(p.title)}</h3>
      <div class="meta">By ${esc(p.author)} • ${p.created_at}</div>
      <p>${esc(p.content).slice(0,220)}${p.content.length>220?"...":""}</p>
      <button onclick="viewPost(${p.id})">Read & Comment</button>
      ${isOwner(p.user_id)?`<button class="edit" onclick="editPost(${p.id})">Edit</button>
      <button class="danger" onclick="deletePost(${p.id})">Delete</button>`:""}
    </article>`).join(""):"<p>No posts yet. Login and create the first post.</p>";
}
function token(){return localStorage.getItem("token")}
function isOwner(id){const u=JSON.parse(localStorage.getItem("user")||"null");return u&&u.id===id}
function openAuth(){hideSections();document.getElementById("auth").classList.remove("hidden")}
function openCreate(){if(!token()){openAuth();return}hideSections();document.getElementById("create").classL
function hideSections(){["feed","create","single","auth"].forEach(x=>document.getElementById(x).classList.ad
function toggleAuth(){
  register=!register;
  authTitle.textContent=register?"Create Account":"Login";
  authAction.textContent=register?"Register":"Login";
  document.querySelector(".link").textContent=register?"Already have account? Login":"Create account";
  name.classList.toggle("hidden",!register);
}
async function submitAuth(){
  const body={email:email.value.trim(),password:password.value};
  if(register)body.name=name.value.trim();
  const r=await fetch(register?"/api/register":"/api/login",{method:"POST",headers:{"Content-Type":"applicat
  const d=await r.json();msg.textContent=d.message;
  if(r.ok&&!register){localStorage.setItem("token",d.token);localStorage.setItem("user",JSON.stringify(d.use
  if(r.ok&&register)toggleAuth();
}
async function createPost(){
  const r=await fetch("/api/posts",{method:"POST",headers:{"Content-Type":"application/json","Authorization"
    body:JSON.stringify({title:postTitle.value,content:postContent.value})});
  const d=await r.json();if(!r.ok)return alert(d.message);
  postTitle.value="";postContent.value="";alert("Post published");home();
}
async function viewPost(id){
  hideSections();single.classList.remove("hidden");
  const r=await fetch("/api/posts/"+id),d=await r.json();
  if(!r.ok)return alert(d.message);
  postDetail.innerHTML=`<h2>${esc(d.post.title)}</h2><div class="meta">By ${esc(d.post.author)} • ${d.post.c
  <p>${esc(d.post.content).replace(/\n/g,"<br>")}</p>
  <h3>Comments</h3>
  ${d.comments.map(c=>`<div class="comment"><b>${esc(c.author)}</b> <span class="meta">${c.created_at}</span
  ${isOwner(c.user_id)?`<button class="danger" onclick="deleteComment(${c.id},${id})">Delete</button>`:""}</
  <textarea id="commentText" placeholder="Write a comment..."></textarea>
  <button onclick="addComment(${id})">Add Comment</button>`;
}
async function addComment(id){
  if(!token()){openAuth();return}
  const content=document.getElementById("commentText").value;
  const r=await fetch("/api/posts/"+id+"/comments",{method:"POST",headers:{"Content-Type":"application/json"
  const d=await r.json();if(!r.ok)return alert(d.message);viewPost(id);
}

async function editPost(id){
  const p=posts.find(x=>x.id===id);if(!p)return;
  const title=prompt("Edit title:",p.title),content=prompt("Edit content:",p.content);
  if(!title||!content)return;
  const r=await fetch("/api/posts/"+id,{method:"PUT",headers:{"Content-Type":"application/json","Authorizati
  const d=await r.json();if(!r.ok)return alert(d.message);loadPosts();
}
async function deletePost(id){
  if(!confirm("Delete this post?"))return;
  const r=await fetch("/api/posts/"+id,{method:"DELETE",headers:{Authorization:"Bearer "+token()}});
  const d=await r.json();if(!r.ok)return alert(d.message);loadPosts();
}
async function deleteComment(cid,pid){
  if(!confirm("Delete this comment?"))return;
  const r=await fetch("/api/comments/"+cid,{method:"DELETE",headers:{Authorization:"Bearer "+token()}});
  const d=await r.json();if(!r.ok)return alert(d.message);viewPost(pid);
}
```

```
function logout(){localStorage.clear();alert("Logged out");home()}
function esc(s){return String(s).replace(/[&<>"']/g,c=>({"&":"&amp;","<":"&lt;",">":"&gt;",'"':"&quot;","'":
home();
```

README.md

```
# Blog Platform with Comments

## Features
- User registration, login and JWT authentication
- Create, edit and delete blog posts
- Read individual blog posts
- Comment section for user interaction
- Users can delete their own comments
- RESTful backend APIs
- SQLite database integration
- Responsive mobile/desktop UI
- Password hashing with bcrypt

## Run
Install Node.js LTS, open this project in VS Code, then run:

npm install
npm start

Open http://localhost:3000

## API
POST /api/register
POST /api/login
GET /api/posts
GET /api/posts/:id
POST /api/posts
PUT /api/posts/:id
DELETE /api/posts/:id
POST /api/posts/:id/comments
DELETE /api/comments/:id

## Security
JWT protects write operations and ownership checks prevent users from editing/deleting other users' posts.
For production, keep the JWT secret in an environment variable and use HTTPS.
```

Submission checklist

✓ User registration, login and authentication
✓ Create, edit and delete blog posts
✓ Comment section
✓ RESTful backend APIs
✓ Database integration
✓ Responsive design
✓ User ownership authorization
✓ README setup instructions